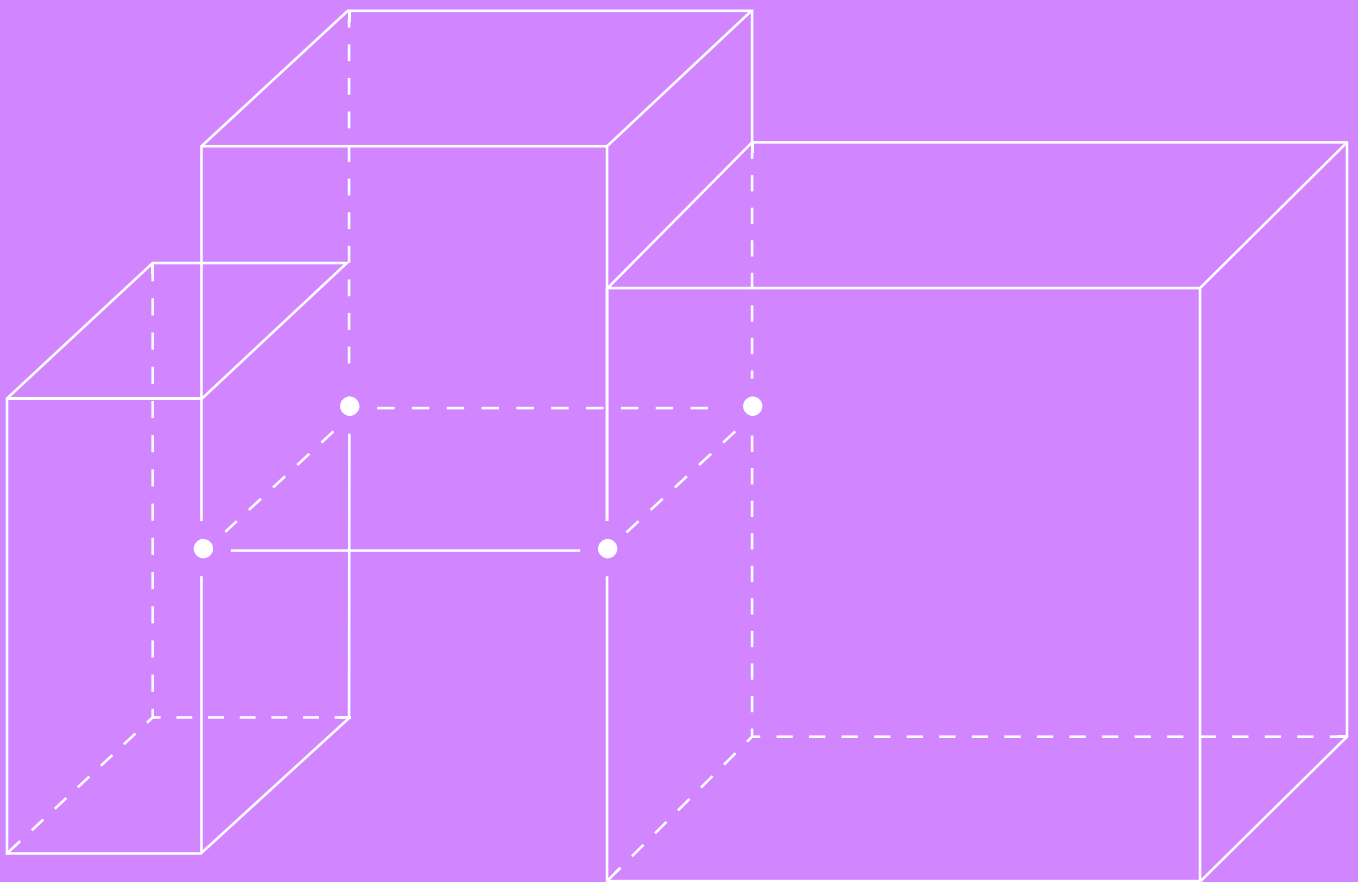


# Агрегация в SQL



**SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)**

**ЛОНГРИД**

**НЕДЕЛЯ 3**

# Введение

При работе с бизнес-задачами часто приходится не просто выгружать сырые данные, но и рассчитывать по ним сводные показатели. Например:

- Сколько заказов было сделано за прошлый месяц?
- Какова общая сумма покупок или средний чек по каждому клиенту?
- В каком году впервые был совершён заказ?

Агрегирующие функции в SQL помогают быстро и эффективно получать ответы на эти вопросы. В этом лонгриде мы рассмотрим принципы работы этих функций и примеры их применения.

## Агрегация данных

### ТЕРМИНЫ

**Агрегация данных** — это объединение данных из нескольких строк в результирующем наборе на основе группировки. Применяется при расчёте метрик, для анализа и создания сводных таблиц.

**Функции агрегации** — это операторы, которые позволяют выполнять расчёты по группам строк и возвращают один результат для каждой группы. Например, они могут суммировать, вычислять средние, минимальные и максимальные значения.

Допустим, у нас есть исходная таблица **orders** с заказами клиентов в интернет-магазине.

| order_id | customer_id | merchant_id | order_amount | order_date | delivery_date | order_status | delivery_status |
|----------|-------------|-------------|--------------|------------|---------------|--------------|-----------------|
| 1        | 1           | 101         | 1000         | 2023-01-01 | 2023-01-03    | paid         | delivered       |
| 2        | 2           | 102         | 2000         | 2023-01-02 | 2023-01-04    | paid         | delivered       |
| 3        | 1           | 103         | 3000         | 2023-01-03 | 2023-01-06    | completed    | in transit      |
| 4        | 3           | 104         | 1500         | 2023-01-04 | 2023-01-05    | completed    | delivered       |
| 5        | 2           | 101         | 5000         | 2023-01-05 | 2023-01-07    | paid         | NULL            |

Рассмотрим работу агрегирующих функций на примере этих данных.

## COUNT

Функция **COUNT** возвращает количество строк в группе.

### Синтаксис

```
SELECT
    COUNT(column1) AS count_value,
    COUNT(DISTINCT column1) AS count_distinct_value
FROM table;
```

#### ПРИМЕР

Найти количество заказов и количество уникальных клиентов.

```
SELECT
    COUNT(order_id) AS total_orders,
    COUNT(DISTINCT customer_id) AS customers
FROM orders;
```

#### Результат

| total_orders | customers |
|--------------|-----------|
| 5            | 3         |

#### Важно:

- можно использовать общий вид **COUNT (\*)** либо синтаксис с указанием конкретного поля — **COUNT(order\_id)**. Ключевое отличие в том, что в первом случае мы посчитаем все строки, включая NULL-значения, а во втором случае пропустим их;
- если нужно посчитать только уникальные значения, пригодится оператор **DISTINCT**. Тогда синтаксис подсчёта будет выглядеть так: **COUNT(DISTINCT order\_id)**;
- если все значения поля содержат NULL, то функция **COUNT(order\_id)** вернёт 0.

## SUM

Функция **SUM** используется для подсчёта общей суммы значений в группе.

### Синтаксис

```
SELECT SUM(column1) AS sum_value
FROM table;
```

**ПРИМЕР**

Найти общую сумму заказов за 2023 год.

```
SELECT SUM(order_amount) AS total_amount
FROM orders
WHERE DATE_PART('year', order_date) = 2023;
```

**Результат**

---

| total_amount |
|--------------|
|--------------|

---

|        |
|--------|
| 12 500 |
|--------|

---

**Важно:**

- если поле содержит NULL-значения, при суммировании они будут проигнорированы;
- если все значения поля содержат NULL, то функция вернёт NULL.

**AVG**

Функция **AVG** рассчитывает среднее значение по группе.

**Синтаксис**

```
SELECT AVG(column1) AS avg_value
FROM table;
```

**ПРИМЕР**

Найти среднюю сумму заказа.

```
SELECT AVG(order_amount) AS avg_amount
FROM orders;
```

**Результат**

---

| avg_amount |
|------------|
|------------|

---

|      |
|------|
| 2500 |
|------|

---

**Важно:**

- аналогично функции **SUM**, функция **AVG** также игнорирует NULL-значения. Это может влиять на точность результатов, если много значений в поле отсутствует;
- если все значения поля содержат NULL, то функция вернёт NULL.

## MIN И MAX

Функции **MIN** и **MAX** находят минимальное и максимальное значение соответственно.

**Синтаксис**

```
SELECT
    MIN(column1) AS min_value,
    MAX(column1) AS max_value
FROM table1;
```

**ПРИМЕР**

Найти минимальный и максимальный размер заказа.

```
SELECT MIN(order_amount) AS min_amount,
       MAX(order_amount) AS max_amount
FROM orders;
```

**Результат**

| min_amount | max_amount |
|------------|------------|
| 1000       | 5000       |

**Важно:**

- если все значения поля содержат NULL, то функции вернут NULL.

## PERCENTILE\_CONT

Функция **PERCENTILE\_CONT** позволяет вычислить значение непрерывного перцентиля.

Она полезна, когда нужно понимать, как распределяются данные (например, средний размер заказа среди 25% лучших клиентов).

## Синтаксис

```
SELECT PERCENTILE_CONT(N) WITHIN GROUP (ORDER BY column)
FROM table;
```

### ПРИМЕР

Найти медианную сумму заказа.

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY
order_amount) AS median_amount
FROM orders;
```

### Результат

| median_amount |
|---------------|
| 2000          |

## Важно:

- аналогично функциям **SUM** и **AVG**, **PERCENTILE\_CONT** также игнорирует NULL-значения.

## GROUP BY

Зачастую требуется вычислить агрегатные значения не по всем данным, а в разрезе определённых групп, например для каждого клиента или каждого товара. В таких случаях данные необходимо сгруппировать с помощью оператора **GROUP BY**, который объединяет строки с одинаковыми значениями в заданных колонках в группы.

## Синтаксис

```
SELECT column1, column2, AGGREGATE_FUNCTION(column3) AS alias
FROM table
GROUP BY column1, column2;
```

**ПРИМЕР**

Подсчитать количество заказов и общую сумму заказов по каждому клиенту.

```
SELECT customer_id,  
       COUNT(order_id) AS total_orders,  
       SUM(order_amount) AS total_amount  
FROM orders  
GROUP BY customer_id;
```

В этом примере мы выбираем все данные из таблицы с заказами, группируем строки с одинаковым значением **customer\_id** с помощью оператора **GROUP BY** и для каждого такого значения вычисляем количество и сумму заказов, указывая это с помощью соответствующих функций в **SELECT**.

**Результат**

| customer_id | total_orders | total_amount |
|-------------|--------------|--------------|
| 1           | 2            | 4000         |
| 2           | 2            | 7000         |
| 3           | 1            | 1500         |

**Важно:**

- группировка может производиться по нескольким полям одновременно, например по дате заказа и клиенту. В таком случае оба поля должны быть указаны и в **SELECT**, и в **GROUP BY**;
- все колонки в **SELECT** либо должны быть указаны в **GROUP BY** как поля для группировки, либо должны быть агрегированы.

## Обработка NULL-значений

Как ты можешь видеть, агрегирующие функции нужно применять с осторожностью, если в данных есть пропуски. На пустых значениях все перечисленные функции, кроме **COUNT**, будут возвращать NULL, а не 0, как можно было бы ожидать.

Если пустые значения в таблице всё же нужно учитывать, их нужно предварительно заменить на ноль с помощью функции **COALESCE**. Она принимает на вход несколько значений и возвращает первое непустое.

## Синтаксис

```
SELECT COALESCE(column1, column2 ...) AS not_null_values  
FROM table;
```

### ПРИМЕР

Для каждого статуса доставки посчитать количество заказов в нём.  
Если статус доставки не заполнен, заменить на 'unknown'.

```
SELECT COALESCE(delivery_status, 'unknown') AS  
delivery_status,  
COUNT(order_id) AS cnt_orders  
FROM orders  
GROUP BY COALESCE(delivery_status, 'unknown');
```

### Результат

| delivery_status | cnt_orders |
|-----------------|------------|
| delivered       | 3          |
| in transit      | 1          |
| unknown         | 1          |



# HAVING

Что делать, если уже после группировки мы хотим оставить только часть полученных результатов? Для этого можно использовать оператор фильтрации **HAVING**.

## Синтаксис

**SELECT**

столбец1,

АГРЕГАТНАЯ\_ФУНКЦИЯ(столбец2) **AS** алиас

**FROM**

таблица

[**WHERE** условие]

**GROUP BY**

столбец1

**HAVING**

условие\_для\_группы;

При этом важно понимать разницу между операторами **WHERE** и **HAVING**.

- **WHERE** фильтрует строки до группировки. Используется, если изначально нужно исключить строки, которые не должны участвовать в дальнейших вычислениях. Не может принимать на вход агрегирующие функции.
- **HAVING** фильтрует группы после их формирования. Используется для фильтрации результатов агрегирующих функций. Может принимать на вход и сами агрегирующие функции, и поля, по которым проводилась группировка.

В прошлом уроке мы уже затронули **очерёдность выполнения операторов** в запросе — она отличается от того, в каком порядке мы выстраиваем синтаксис:



**ПРИМЕР**

Найти клиентов, у которых общая сумма заказов с 1 января 2023 года превышает 5000.

```
SELECT customer_id,  
       SUM(order_amount) AS total_amount  
FROM orders  
WHERE order_date >= '2023-01-01'  
GROUP BY customer_id  
HAVING SUM(order_amount) > 5000;
```

В данном примере мы сначала выбираем заказы по условию на даты с помощью оператора **WHERE**, затем группируем заказы по клиентам с помощью оператора **GROUP BY** и уже после группировки оставляем только тех клиентов, у которых сумма больше 5000, используя для этого оператор **HAVING**.

Результат

| customer_id | total_amount |
|-------------|--------------|
| 2           | 7000         |

## Агрегирующие функции с условиями

Иногда требуется посчитать агрегированное значение не на всех данных, а только на определённой их части. Для этого можно использовать операторы **FILTER** или **CASE WHEN** — оба они позволяют применить агрегирующую функцию только к тем строкам, которые удовлетворяют условию.

Синтаксис **FILTER**

```
SELECT AGGREGATE_FUNCTION(column1) FILTER (WHERE condition1) AS  
agg_column1  
FROM table;
```

```

SELECT
    AGGREGATE_FUNCTION(
        CASE
            WHEN condition1 THEN column1 [ELSE value1]
        END
    ) AS agg_column1
FROM table;

```

## ПРИМЕР

Посчитать сумму заказов со статусом 'completed' и 'paid'.

```

SELECT
    SUM(order_amount) FILTER (WHERE delivery_status =
'completed') AS completed_amount,
    SUM(order_amount) FILTER (WHERE delivery_status =
'paid') AS paid_amount
FROM orders;

```

В данном примере мы могли бы применить фильтрацию ко всей таблице внутри оператора **WHERE**, но тогда эти два значения пришлось бы посчитать двумя отдельными запросами. С помощью **FILTER** мы сделали это сразу, непосредственно при подсчёте агрегатных значений.

Аналогичный результат можно получить и с помощью конструкции **CASE WHEN**.

```

SELECT
    SUM(CASE WHEN delivery_status = 'completed' THEN
order_amount END) AS completed_amount,
    SUM(CASE WHEN delivery_status = 'paid' THEN
order_amount END) AS paid_amount
FROM orders;

```

Результат

| completed_amount | paid_amount |
|------------------|-------------|
| 4500             | 8000        |

### Важно:

- различий в производительности у этих двух подходов практически нет.  
Основное различие в том, что внутри **CASE WHEN** при необходимости можно собрать не просто фильтрацию, а более сложную логическую конструкцию;
- оба подхода могут использоваться вместе с **GROUP BY** для группировки данных.

#### ПРИМЕР

Посчитать сумму заказов со статусом 'completed' для каждого клиента.

```
SELECT
    customer_id,
    SUM(order_amount) FILTER (WHERE delivery_status =
'completed') AS completed_amount
FROM orders
GROUP BY customer_id;
```

#### Результат

| customer_id | completed_amount |
|-------------|------------------|
| 1           | 3000             |
| 2           | NULL             |
| 3           | 1500             |

Откуда появилось NULL-значение? Вспомним: если все значения суммирования оказываются пустыми, функция возвращает NULL. Таким образом, для тех клиентов, у которых нет заказов с заданным статусом, мы получаем пустой результат.

Перепишем запрос с использованием оператора **CASE WHEN**.

```
SELECT
    customer_id,
    SUM(CASE WHEN delivery_status = 'completed' THEN
order_amount ELSE 0 END) AS completed_amount
FROM orders
GROUP BY customer_id;
```

В данном примере мы использовали дополнительное условие для тех строк, где статус не равен 'completed', заменяли такие значения на ноль с помощью **ELSE 0**. Теперь, суммируя нулевые значения, в итоговой таблице мы также получим ноль.

| customer_id | completed_amount |
|-------------|------------------|
| 1           | 3000             |
| 2           | 0                |
| 3           | 1500             |

## Заключение

В этом материале мы разобрали, как использовать агрегацию для получения сводных данных.

Теперь посчитать общие продажи компании или найти самых прибыльных клиентов не составит труда!